实验二　对称密码应用

姓名	专业年级	课程名称	实验时间
姚嫣如	23 网络空间安全	网络安全技术	2026.4.3

一、实验目的

1. 理解传统**对称密码（维吉尼亚密码）**的原理及其安全性缺陷。
2. 掌握通过**频率分析**和**卡西斯基试验**（Kasiski Examination）破解维吉尼亚密码的方法。
3. 熟悉**现代对称加密算法**（AES-GCM, ChaCha20-Poly1305）的特点及应用场景。
4. 学习使用 Python 的 socket 库进行网络编程，并结合 cryptography 库实现加密通信。

二、实验环境

- 操作系统：物理机（Windows） + 虚拟机（Ubuntu）
- 开发语言： Python 3.13
- 依赖库： cryptography

三、实验内容

1、维吉尼亚密码破解

（1） 实验目标： 破解给定的密文，找出密钥，并编写解密程序。

（2） 实验思路:

- 使用卡西斯基试验确定密钥长度。
- 利用重合指数IC验证密钥长度。
- 对密文按密钥长度进行分组，利用字母频率分析（Frequency Analysis）确定每一位的密钥。
- 编写解密逻辑还原明文。

2、文件加密传输

（1）实验目标： 编写服务端（虚拟机）和客户端（物理机），通过加密套接字传输文件。

（2）实验思路：

- 使用 cryptography 库生成/共享 AES 或 ChaCha20 密钥。
- 物理机读取文件，使用选定算法加密，通过 Socket 发送。
- 虚拟机监听端口，接收数据，解密并保存文件。

四、实验步骤

（一）维吉尼亚密码破解

1、维吉尼亚密码

（1）**多表替换密码**（Polyalphabetic Substitution Cipher）

在16世纪它被发明后，曾被称为“不可破译的密码”，直到19世纪中叶才被系统性地破解。相较于单表替换（如凯撒密码），它使用一个**单词（密钥）**来决定位移。密钥中的每个字母代表不同的位移量，同一个明文字母在密文中可能对应不同的字母,从而打破了明文的频率特征。

（2）维吉尼亚方格

通常使用一张 26×26 的字母表格（也叫 Tabula Recta）来进行操作。

- 第一行是位移为0的字母表（A-Z）
- 第二行是位移为1的字母表（B-Z-A）
- 以此类推，共有26行

	A	**B**	**C**	**D**	**E**	...
A	A	B	C	D	E	...
B	B	C	D	E	F	...
C	C	D	E	F	G	...
D	D	E	F	G	H	...

（3）加密原理

假设我们要加密：

- 明文 (Plaintext): HELLO
- 密钥 (Key): KEY

步骤：

① 重复密钥： 将密钥重复，直到与明文长度一致。

- 明文：H E L L O
- 密钥：K E Y K E

② 查找对应：

- 第一个字母：明文 H，密钥 K。在表格中找到 H行 和 K列 的交点，结果是 R。
- 第二个字母：明文 E，密钥 E。在表格中找到 E行 和 E列 的交点，结果是 I。

以此类推...

（4）数学表达式

- 加密公式：

$$C\,i\, = (P\,i\, + K\,i\,)mod26$$

- 解密公式：

$$P\,i\, = (C\,i\, - K\,i\, + 26)mod26$$

（5）破解方法

- 弱点：**周期性**，如果密钥长度是 7，那么每隔 7 个字母，加密规则就会重复。

① **卡西斯基试验**（Kasiski Examination）： 寻找密文中重复出现的片段（比如重复的单词），计算它们之间的距离。这些距离的公约数很可能就是密钥的长度。

② 重合指数IC：随机乱码中字母出现的概率是均匀的，而自然语言（如英语）中字母出现的概率是不均匀的。

② 频率分析（分组）： 一旦知道了密钥长度（假设是 L），就把密文分成 L 组。每一组内的字母都是用同一个位移加密的（退化成了凯撒密码），此时再对每一组分别使用频率分析，就能轻而易举地找出密钥。

2、密钥分析

Hmnn qf vcvav rdag jcijk. Qc qf jcckqqwo n zclarbumfa, dlkaqwo bv. Mun tiamabq lezm xv gppoqkp da, nvb wa aqut yqte kr bqzbceh kyz lpvtbrar. Bqm vvqtexccqbvq wa fcrtq mldqvm, jvq ec wept nvqcpe plzxctp rhaq. Bqm omyupc en nnafikr kjvawr ba hqvurl zy ziicp. Bcp fhiaq ung nenmaq, whz fajha fqyt uiplma, jhb rhwx eqqpp rhac kamnb gn xiidbl ild cswmvraq ajh bacgp jiria xv swp ahp brur bm ckqm.

Zs vxb fxcnz evm enare usca tvdcs wgkduhtytero xjwmato xpjb jqjl krth bhzl tk hcbb nvb aolmb. Xhase jsb bw zcah plm vigmpiwp ib bum gdaet, owe qbewpa jtbvc ijzmbb yqde smbq urilijk iwl nzc ob ivmceqlg sszcp. Nlb lkzm cw n pmuoi iwl lws hwzm j pbuc. Azh zroubcoqwvnaf bm a ymbh ial woq liem n kmmiyvrbl. Ibd pvccp gw y pepm xn emb bnmkt ial woq liem n aahkst. Jlq zclekqxv gw rha lcktrar ob ilrnvkcs wrl hwh pyva e ajvpbsanc. Iml wcqtegm cw gpc fwv nucao poqrl xn uckaj ivmmndmr wrl hwh pyva gqeepqiigqmn. Lyb cpru ylh xwpmgpcr, abiub gpcm wfwem gpcin tznarvr iitmanrkrikra, jlq bm tdiu cpr dgsesv xn uckajoqwl embeaqmm, nbzcvav namr wd nail jvq arrejm jvq gmu dedn i scruni troubcd smbq bum pazmiwb pwjonw wo pbxc.

（1）确定密钥长度L

- 重复字母串：Bqm、jvq、qv、wo ……

👉 初步判定：密钥长度 ≈ 7 ~ 9

（2）通过重合指数 IC 确定密钥长度

- 随机文本 IC ≈ **0.038**
- 正常英文 IC ≈ **0.065**

  关键代码如下：

```python
# 预处理
def clean_text(text):
    return re.sub('[^A-Z]', '', text.upper())


# IC计算
from collections import Counter

def calculate_ic(text):
    N = len(text)
    freq = Counter(text)

    ic = 0
    for f in freq.values():
        ic += f * (f - 1)

    return ic / (N * (N - 1)) if N > 1 else 0

```

```python
# 按密钥长度分组
def split_by_key_length(text, key_len):
    groups = ['' for _ in range(key_len)]

    for i, c in enumerate(text):
        groups[i % key_len] += c

    return groups

# 计算不同密钥长度的IC
def find_key_length(ciphertext, max_len=20):
    ciphertext = clean_text(ciphertext)

    result = []

    for key_len in range(1, max_len + 1):
        groups = split_by_key_length(ciphertext, key_len)

        ic_values = [calculate_ic(g) for g in groups]
        avg_ic = sum(ic_values) / len(ic_values)

        result.append((key_len, avg_ic))

    return result

results = find_key_length(cipher, 20)

for k, ic in results:
    print(f"key_len={k}, IC={ic:.4f}")
```

运行结果如下：

```
key_len=1, IC=0.0444
key_len=2, IC=0.0440
key_len=3, IC=0.0460
key_len=4, IC=0.0440
key_len=5, IC=0.0436
key_len=6, IC=0.0453
key_len=7, IC=0.0444
key_len=8, IC=0.0440
key_len=9, IC=0.0489
key_len=10, IC=0.0428
key_len=11, IC=0.0449
key_len=12, IC=0.0446
key_len=13, IC=0.0442
key_len=14, IC=0.0435
key_len=15, IC=0.0455
key_len=16, IC=0.0429
key_len=17, IC=0.0430
key_len=18, IC=0.0487
key_len=19, IC=0.0409
key_len=20, IC=0.0420
```

🍁 没有看到预期的接近0.065附近的IC值：

- 可能原因 1：文本并非纯正的英语自然语言（小说/新闻）。
- 可能原因2：维吉尼亚密码本质是多个凯撒密码叠加：
  - 密钥越长 → 混合程度越高
  - 字母分布越接近随机
- 可能原因3：分组后文本长度不足

  当密钥长度较大（如本题为 9）时，每组文本长度 = 原长度 ÷ 9，每组样本数减少

  导致统计不稳定，字母频率偏差较大，从而使 IC 值**偏低**

虽然未接近0.065，但在key_len = 9 / 18，可以看到小峰值，从而猜测密钥长度 = 9

- 可能原因4：两段原文为不同密钥

👉 尝试分段破解：

结果如下：

```
key_len=1, IC=0.0474
key_len=2, IC=0.0486
key_len=3, IC=0.0510
key_len=4, IC=0.0482
key_len=5, IC=0.0452
key_len=6, IC=0.0515
key_len=7, IC=0.0457
key_len=8, IC=0.0508
key_len=9, IC=0.0641
key_len=10, IC=0.0453
key_len=11, IC=0.0435
key_len=12, IC=0.0493
key_len=13, IC=0.0479
key_len=14, IC=0.0524
key_len=15, IC=0.0522
```

```
key_len=1, IC=0.0438
key_len=2, IC=0.0435
key_len=3, IC=0.0453
key_len=4, IC=0.0434
key_len=5, IC=0.0432
key_len=6, IC=0.0440
key_len=7, IC=0.0439
key_len=8, IC=0.0449
key_len=9, IC=0.0535
key_len=10, IC=0.0429
key_len=11, IC=0.0454
key_len=12, IC=0.0428
key_len=13, IC=0.0453
key_len=14, IC=0.0426
key_len=15, IC=0.0440
key_len=16, IC=0.0428
key_len=17, IC=0.0403
key_len=18, IC=0.0524
key_len=19, IC=0.0385
key_len=20, IC=0.0450
```

长度都为9。

（3）还原密钥—频率分析

Chi-squared test（卡方检验）

- 枚举每个可能位移（0~25），计算解密后文本的字母分布，与“标准英文频率”对比

  误差最小 → 最可能的密钥

关键代码如下：

```
# 英文字母频率（标准）
ENGLISH_FREQ = {
```

```
    'A': 0.08167, 'B': 0.01492, 'C': 0.02782, 'D': 0.04253,
    'E': 0.12702, 'F': 0.02228, 'G': 0.02015, 'H': 0.06094,
    'I': 0.06966, 'J': 0.00153, 'K': 0.00772, 'L': 0.04025,
    'M': 0.02406, 'N': 0.06749, 'O': 0.07507, 'P': 0.01929,
    'Q': 0.00095, 'R': 0.05987, 'S': 0.06327, 'T': 0.09056,
    'U': 0.02758, 'V': 0.00978, 'W': 0.02360, 'X': 0.00150,
    'Y': 0.01974, 'Z': 0.00074
}

# 预处理
def clean_text(text):
    return re.sub('[^A-Z]', '', text.upper())

# 分组
def split_by_key_length(text, key_len):
    groups = ['' for _ in range(key_len)]
    for i, c in enumerate(text):
        groups[i % key_len] += c
    return groups

# 凯撒解密（单组）
def caesar_decrypt(text, shift):
    result = ""
    for c in text:
        p = (ord(c) - ord('A') - shift) % 26
        result += chr(p + ord('A'))
    return result

# 计算卡方值
def chi_squared(text):
    N = len(text)
    freq = Counter(text)

    chi2 = 0
    for letter in string.ascii_uppercase:
        observed = freq.get(letter, 0)
        expected = ENGLISH_FREQ[letter] * N
        if expected > 0:
            chi2 += (observed - expected) ** 2 / expected
    return chi2

# 找最佳shift（关键函数）
def find_best_shift(group):
    min_chi = float('inf')
    best_shift = 0

    for shift in range(26):
```

```
        decrypted = caesar_decrypt(group, shift)
        chi = chi_squared(decrypted)

        if chi < min_chi:
            min_chi = chi
            best_shift = shift

    return best_shift

# 还原密钥
def get_key(ciphertext, key_len):
    ciphertext = clean_text(ciphertext)
    groups = split_by_key_length(ciphertext, key_len)

    key = ""
    for g in groups:
        shift = find_best_shift(g)
        key += chr(shift + ord('A'))

    return key

```

结果如下：

```
Key = WEIJINIYA
```

（4）验证破解

代码如下：

```
def vigenere_decrypt(ciphertext, key):
    alphabet = "abcdefghijklmnopqrstuvwxyz"
    plaintext = ""
    key = key.lower()
    key_index = 0

    for char in ciphertext:
        if char.lower() in alphabet:
            c_idx = alphabet.find(char.lower())
            k_idx = alphabet.find(key[key_index % len(key)])

            # 解密公式: P = (C - K) mod 26
            p_idx = (c_idx - k_idx) % 26

            res_char = alphabet[p_idx]
            plaintext += res_char if char.islower() else res_char.upper()
            key_index += 1
```

```
        else:
            plaintext += char
    return plaintext

```

结果如下：

- 完整破解

```
最终确认密钥: weijiniya
------------------------------

Life is never just being. It is becoming a relentless, flowing on. Our parents live on through
us, and we will live on through our children. The institutions we build endure, and we will endure
through them. The beauty we fashion cannot be dimmed by death. Our flesh may perish, our hands will
wither, but that which they creat in beauty and goodness and truth lives on for all time to come.
Qk ipd fbyfq wie gnenw lkps vvhyk nyxvwhxulvjb plwqwlf pcbd jufd bjgz dhdh lb zptd nzx sfdzt. Zheow
ako ty zgwz gdz nkgqlanh vt duq cvrwg, gye uxwnhn bvbzy aarztd yuzw jeoi wrmhaac von ndy gs
aieeeuhy jkmur. Npx dbrz uy n timfa von lao znrz b rbyy. Sqz mjqufyghoifcf fi s peoz kap sgh
dvwo n oiezqijdl. Mxv gnpur ga u hvhz pp eqx teexl kap sgh dvwo n ewzbkg. Bnq dydvcdpx ga nzr
dpcvren gs ayjpvoyk njy zyh tunr w nbxpfoseu. Ven wgmlvyz uy gty xnn ameas lghjy pp uggsa aieonhij
njy zyh tunr ydwgpueaxizf. Nyf yhim ldj xalexhpj, cbmqt xhpe yfaae xhpap tdjsine aktqwficeamre, fdh
tz lfiy yhi vtkgsz tf luxslousd veowcqqi, fsrpncv rwei oq fcip fnh sejgjq fnh yzm fehj a juempi
xngltpv umfm tle csbmmst gowgpw ak hspp.
```

- 分段破解

```
最终确认密钥: weijiniya
------------------------------
Life is never just being. It is becoming a relentless, flowing on. Our parents live on through
us, and we will live on through our children. The institutions we build endure, and we will endure
through them. The beauty we fashion cannot be dimmed by death. Our flesh may perish, our hands will
wither, but that which they creat in beauty and goodness and truth lives on for all time to come.
```

```
最终确认密钥: weijiniya
------------------------------

Do not spend and waste your lives accumulating objects that will only turn to dust and ashes. Pusue
not so much the material as the ideal, for ideals alone invest life with meaning and are of
enduring worth. Add love to a house and you have a home. Add righteousness to a city and you
have a community. Add truth to a pile of red brick and you have a school. Add religion to the
hublest of edifices and you have a sanctuary. Add justice to the far flung round of human endeavor
and you have civwcikikmee.
```

```
最终确认密钥: weijiniya
------------------------------

 Put them all together, exalt them above their present imperfections, add
to them the vision of humankind redeemed, forever free of need and strife and you have a future
lighted with the radiant colors of hope.
```

🌏 密钥相位（Key Phase）/ 密钥对齐问题。

第一段的总字母数并非密钥长度（9）的整数倍，导致解密程序在进入第二段时，密钥的指向位置（相位）发生了偏移。而实验结果证实，该密文的三段是**独立加密**的，即每一段的首字母都对应密钥的首字母 W。

（二）文件加密传输

1. 安装相关加密库
    - 打开虚拟机，并用物理机连接到虚拟机

```
C:\Users\RMX08> ssh yolanda@lab
The authenticity of host 'lab (192.168.126.128)' can't be established.
ED25519 key fingerprint is SHA256:/e3CAYMUEvz+rV8JxogefGqMwDh09WN/9JShwMI4GZc.
This host key is known by the following other names/addresses:
    C:\Users\RMX08/.ssh/known_hosts:1: 192.168.126.128
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'lab' (ED25519) to the list of known hosts.
yolanda@lab's password:
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.19.0-32-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

 * Introducing Expanded Security Maintenance for Applications.
   Receive updates to over 25,000 software packages with your
   Ubuntu Pro subscription. Free for personal use.

     https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri Apr 10 17:23:35 2026 from 192.168.126.1
yolanda@yyr:~$
```

    - 在物理机和虚拟机上分别打开终端，安装加密库

```
C:\Users\RMX08>pip install cryptography
Collecting cryptography
  Downloading cryptography-46.0.7-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp313-cp313-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-46.0.7-cp311-abi3-win_amd64.whl (3.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.5/3.5 MB 208.5 kB/s eta 0:00:00
Downloading cffi-2.0.0-cp313-cp313-win_amd64.whl (183 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-46.0.7 pycparser-3.0
```

    先给虚拟机安装pip，再通过pip安装加密库

```
yolanda@yyr:~$ sudo apt install python3-pip
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

```
yolanda@yyr:~$ pip3 install cryptography
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: cryptography in /usr/lib/python3/dist-packages (3.4.8)
```

2. 在物理机上实现文件加密并发送

🍁 加密算法：AES-GCM

目前全球最广泛使用的对称加密算法，**GCM (Galois/Counter Mode)** 是一种工作模式，它具备 **AEAD**（带身份验证的加密）特性：

- **机密性：** 使用计数器模式加密数据。
- **完整性：** 生成一个“认证标签”（Tag）。如果数据在传输过程中被修改（哪怕是一个比特），解密端在校验 Tag 时会直接报错，拒绝解密。
- **Nonce (随机数)：** GCM 要求每次加密必须使用一个唯一的随机数（通常为 12 字节）。**Nonce 必须随密文一起发送给接收方**，否则无法解密。

代码如下：

```python
import socket
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# 1. 预共享密钥 (必须与接收端一致)
SHARED_KEY = b"this_is_a_very_secret_32_byte_key"

def start_sender(vm_ip):
    # 要发送的文件
    file_path = "test_file.txt"
    if not os.path.exists(file_path):
        with open(file_path, "w") as f: f.write("Hello, this is a secret file!")

    # 2. 读取文件并加密
    with open(file_path, "rb") as f:
        file_data = f.read()

    aesgcm = AESGCM(SHARED_KEY)
    nonce = os.urandom(12) # 生成随机的 12 字节 Nonce
    ciphertext = aesgcm.encrypt(nonce, file_data, None)

    # 3. 组合数据并发送 (Nonce + Ciphertext)
    payload = nonce + ciphertext

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        client_socket.connect((vm_ip, 9999))
        client_socket.sendall(payload)
        print("加密文件已成功发送！")
    finally:
        client_socket.close()

```

```python
if __name__ == "__main__":
    # 请在此处替换为你虚拟机的实际 IP 地址
    VM_IP = "192.168.126.128"
    start_sender(VM_IP)
```

3. 在虚拟机上实现接收数据并解密

代码如下：

```python
import socket
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# 1. 预共享密钥 (必须是 16, 24 或 32 字节)
# 注意：物理机和虚拟机必须使用完全相同的 Key
SHARED_KEY = b"this_is_a_very_secret_32_byte_key"

def start_receiver():
    # 创建 Socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(('0.0.0.0', 9999)) # 监听所有接口的 9999 端口
    server_socket.listen(1)

    print("等待物理机连接...")
    conn, addr = server_socket.accept()
    print(f"连接来自: {addr}")

    # 接收数据
    # AES-GCM 的密文结构通常为: Nonce(12字节) + Ciphertext
    data = conn.recv(1024 * 1024) # 接收最大 1MB 数据
    conn.close()

    # 2. 拆分 Nonce 和密文
    nonce = data[:12]
    ciphertext = data[12:]

    # 3. 解密
    aesgcm = AESGCM(SHARED_KEY)
    try:
        decrypted_data = aesgcm.decrypt(nonce, ciphertext, None)
        # 保存解密后的文件
        with open("received_file.txt", "wb") as f:
            f.write(decrypted_data)
        print("文件接收成功并已安全解密！保存为 'received_file.txt'")
    except Exception as e:
        print(f"解密失败: {e} (数据可能已被篡改或密钥错误)")

```

```
38 if __name__ == "__main__":
39     start_receiver()
```

先运行receiver

```
yolanda@yyr:~$ cd Desktop/lab/lab2
yolanda@yyr:~/Desktop/lab/lab2$ ls
receiver.py
yolanda@yyr:~/Desktop/lab/lab2$ python3 receiver.py
等待物理机连接...
```

再运行server

```
"D:\yyr\study\aa 专业课\网络安全技术\.
加密文件已成功发送!
```

```
yolanda@yyr:~/Desktop/lab/lab2$ python3 receiver.py
等待物理机连接...
连接来自: ('192.168.126.1', 63799)
文件接收成功并已安全解密! 保存为 'received_file.txt'
```

4. 查看解密后内容

```
yolanda@yyr:~/Desktop/lab/lab2$ ls
received_file.txt  receiver.py
yolanda@yyr:~/Desktop/lab/lab2$ cat received_file.txt
Hello, this is a secret file!yolanda@yyr:~/Desktop/lab/lab2$
```

五、实验心得

通过本次“对称密码应用”实验，我不仅在理论上深入理解了对称加密算法的演进，更在实践中掌握了密码分析与安全编程的核心技能。

在破解维吉尼亚密码的过程中，我深刻体会到“统计规律”是传统加密算法的死穴。通过计算重合指数（IC），我发现即使密文看起来杂乱无章，数学特征依然能准确地暴露出密钥的长度。虽然在实际分析中由于多段落独立加密的原因，自动化工具出现了偏差，但通过“频率分析+人工逻辑推理”的结合，最终成功破译。

在实验二中实现 AES-GCM 加密传输时，我遇到了 ValueError 密钥长度报错。这让我意识到现代密码学协议的严谨性：算法对输入参数（如 16/32 字节密钥、12 字节 Nonce）有着严格的规范，差一个比特都会导致系统崩溃。AES-GCM 的 AEAD 特性不仅保护了数据的机密性，还提供了完整性校验，这种“防御性编程”的思想是构建安全网络应用的基础。

本次实验涉及物理机与虚拟机的 Socket 通信，过程中遇到的网络配置、pip 环境安装以及 scp 文件传输等细节，锻炼了我解决实际工程问题的能力。